FUNDAMENTOS DE APRENDIZAGEM DE MÁQUINA

Maikon Lucian Lenz



SAGAH SOLUÇÕES EDUCACIONAIS INTEGRADAS

Aprendizagem não supervisionada: aplicação do *k-means*

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Processar um conjunto de dados para a tarefa de agrupamento (*clustering*).
- Empregar o algoritmo *k-means* para uma tarefa de agrupamento.
- Aplicar métricas para avaliar uma formação dos *clusters*.

Introdução

A aplicação de algoritmo de agrupamento depende de uma seleção adequada dos dados, e inúmeras ferramentas foram desenvolvidas para facilitar a implementação desses dados, com destaque para a biblioteca SciKit-Learn para Python. A biblioteca inclui métodos e classes configuráveis para todas as abordagens de aprendizado. O algoritmo de *k-means* permite a configuração de diversos parâmetros que influenciarão não apenas os *clusters* encontrados, mas também a velocidade com que os dados são processados e a quantidade de memória necessária.

Neste capítulo, você verá como dados podem ser carregados para variáveis com o auxílio das bibliotecas de SciKit-Learn, organizados e manipulados a partir de bibliotecas como Pandas e Seaborn. Também serão treinados diferentes conjuntos de dados utilizando o algoritmo de *k-means*, incluso nessas bibliotecas, e as métricas disponíveis para avaliar a eficiência dos agrupamentos realizados.

1 Ferramentas e dados

Existem muitas linguagens de programação e ferramentas adequadas para todo o tipo de aplicação. Você pode optar pela linguagem de sua preferência para desenvolver sistemas que abordem algum tipo de aprendizado de máquina. No entanto, a linguagem Python é uma das primeiras opções entre os desenvolvedores e pesquisadores quando o assunto é análise e processamento de dados.

Devido à sintaxe simplificada e de fácil aprendizado, o Python conquistou espaço significativo em trabalhos acadêmicos, a maioria deles sobre análise de dados. Assim, surgiram muitas bibliotecas e ferramentas associadas ao Python que permitiram a manipulação cada vez mais facilitada de dados.

Em aprendizado de máquina, uma das bibliotecas mais importantes, especialmente para aqueles que estão iniciando na área, é a SciKit-Learn, totalmente voltada para problemas de aprendizado de máquina. Ela conta com ferramentas de pré-processamento de dados, interpretação de arquivos, criação de gráficos e outras formas de exibição das informações.

Também estão inclusos na biblioteca SciKit-Learn conjuntos de dados úteis para todos os tipos de abordagens de aprendizado de máquina, com destaque para: o conjunto íris, com 150 amostras de três espécies de flores do gênero íris em que, para cada uma, são atribuídos o comprimento e a largura de pétalas e sépalas, além da espécie em si, utilizado para modelos de classificação; e o conjunto *Boston house-prices* (preço de casas em Boston), utilizado para modelos de regressão, com 506 amostras de casas acompanhadas do atributo alvo preço e de 13 atributos previsores que incluem a taxa de criminalidade *per capita* da região, o número de quartos, entre outros.

Outras bibliotecas bastante úteis na análise de dados são: Numpy, com recursos que facilitam o cálculo e a manipulação de vetores, além de servir de base para o SciKit-Learn; Pandas, para criar estruturas de dados de fácil visualização e integração; Matplotlib e uma extensão chamada Seaborn, ambas para criação de gráficos.

Com todos esses recursos, você será capaz de: carregar dados de diferentes fontes, arquivos, páginas da *web* e conjuntos de dados embutidos nas bibliotecas; criar conjuntos aleatórios; visualizar e analisar os dados recebidos, além de suas estatísticas que serão úteis para determinar qual é a melhor abordagem a seguir no aprendizado de máquina, bem como para adequá-los aos próximos passos, por exemplo, ajustando a escala dos atributos para que todos contribuam igualmente no processo de treinamento; testar diferentes algoritmos de aprendizado e o impacto que mudanças no conjunto de dados

e seus respectivos parâmetros têm sobre o modelo escolhido; exibir as informações utilizando gráficos e recursos visuais de forma facilitada.

Todos os exemplos apresentados e as bibliotecas mencionadas podem ser utilizados no ambiente de desenvolvimento do Jupyter, que pode ser facilmente instalado por meio da plataforma Anaconda. A instalação das bibliotecas pode ser feita pelo *prompt* de comando do Anaconda utilizando o comando `conda install package`, em que o termo `package` deve ser substituído pela biblioteca desejada. As bibliotecas aqui utilizadas podem ser instaladas por meio de um único comando: `conda install scikit-learn pandas seaborn matplotlib numpy`. Concluídas as instalações, o Jupyter pode ser iniciado pelo comando: `jupyter notebook`. O exemplo a seguir demonstra de que forma um conjunto de dados da biblioteca SciKit-Learn é carregado.



Exemplo

```
# Exemplo 1
# importa o módulo que contém os conjuntos de dados
from sklearn import datasets as ds
# carrega o conjunto iris de dados
iris = ds.load_iris()
```

A biblioteca SciKit-Learn é dividida em módulos, e o módulo `datasets` é destinado, exclusivamente, aos conjuntos de dados embarcados na biblioteca. O caractere # indica que o restante da linha será ignorado pelo compilador, servindo apenas de comentário. A primeira linha de instrução importa o módulo `datasets` existente na biblioteca SciKit-Learn (`sklearn`), que passará a ser referenciado somente pelo nome `ds`. Na sequência, a variável `iris` recebe um dicionário contendo o conjunto de dados íris através do método `load_iris()`. A variável `iris` contém seis itens:

- `data` — matriz ou vetor em que cada linha representa uma instância, e as colunas são diferentes atributos de uma mesma instância;
- `target` — índice para os rótulos de cada instância na mesma ordem em que aparecem em data;

- `target_names` — os nomes dados a cada um dos rótulos;
- `feature_names` — nomes de cada uma das colunas de atributos;
- `DESCR` — um texto descrevendo autoria e características do conjunto de dados;
- `filename` — caminho em que se encontra o arquivo com os dados.

Essa disposição não é a mais prática para visualizar e analisar os dados. Então, é possível recorrer à biblioteca Pandas e criar um objeto `DataFrame`, que se parece com uma planilha, conforme o exemplo a seguir.



Exemplo

```
# Exemplo 2
# importa a biblioteca pandas
import pandas as pa
# cria um DataFrame a partir do objeto íris
df     =     pa.DataFrame(data=iris['data'],
columns=iris['feature_names'])
# adiciona ao DataFrame a coluna de atributos alvo
dfT = df.copy()
dfT['target'] = iris['target']
display(df), display(dfT)
```

O parâmetro `columns` utiliza o vetor `feature_names` para nomear cada uma das colunas de atributos. Já o parâmetro `data` pode receber todos os valores de atributos previsores e do atributo alvo, para isso, basta que os vetores sejam concatenados antes da criação do `DataFrame`. No entanto, adicionar essa coluna do atributo alvo separadamente é mais simples e legível. O resultado dessa transformação pode ser visualizado no Quadro 1.

Quadro 1. Quadro de dados utilizando a biblioteca Pandas e o conjunto de dados íris do SciKit-Learn

	sepal length (cm)	sepal width (cm)	petal lenght (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
…	…	…	…	…	…

Observe que o conjunto de dados íris contém quatro atributos previsores que especificam tamanhos de sépala e pétala, que são: *sepal length* (comprimento da sépala); *sepal width* (largura da sépala); *petal length* (comprimento da pétala); e *petal width* (largura da pétala). Todos os atributos previsores são dados em centímetros. O Quadro 1 exibe apenas as cinco primeiras instâncias do conjunto em que cada linha representa uma delas com seus respectivos valores para cada atributo.

O atributo denominado *target* é o atributo alvo, e os seus valores indicam uma das três classes possíveis de flor que a instância pode pertencer: *setosa* (0), *versicolor* (1) e *virginica* (2). Esse atributo, no entanto, não será utilizado no treinamento de tarefas de agrupamento, já que esse tipo de tarefa é de aprendizado não supervisionado e visa encontrar *clusters* que descrevam características intrínsecas dos dados.

Outra vantagem de se utilizar o `DataFrame` é o uso do método `describe`, que exibe um resumo com estatísticas a respeito de cada coluna de atributos (Quadro 2). Essas informações poderão guiá-lo na escolha da abordagem de aprendizado a ser utilizada e na necessidade da preparação e adequação dos dados.

Quadro 2. Resumo estatístico do `DataFrame` do conjunto de dados íris

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

A biblioteca Pandas conta ainda com diversos métodos de leitura de dados a partir de outras fontes, por exemplo:

- `read_clipboard` — cria um `DataFrame` a partir dos dados copiados na prancheta (*clipboard*) do sistema operacional;
- `read_csv` — arquivos em que colunas (atributos) são separadas por algum caractere especial, geralmente uma vírgula, e cada linha representa uma nova instância;
- `read_html` — varre uma página na *web* na busca por tabelas que possam conter dados;
- `read_json` — formato textual para representação de dados de qualquer tipo;
- `read_sql` — leitura de banco de dados SQL, entre outros.

Além disso, é possível criar conjuntos de dados com valores aleatórios utilizando métodos como o `make_blobs` da biblioteca SciKit-Learn. Nos parâmetros, devem ser especificados a quantidade de instâncias (`n_samples`), a quantidade de atributos (`n_features`), um número aleatório para servir de base (`random_state`), o desvio padrão que os dados devem apresentar (`cluster_std`) e outros menos usuais. O exemplo a seguir mostra como criar um conjunto de dados de 100 instâncias com quatro atributos.



Exemplo

```
# Exemplo 3:
from sklearn import datasets as ds
mb = ds.make_blobs(n_samples=100, n_features=4)
```

O resultado é uma tupla em que o primeiro item é uma matriz contendo os valores de cada um dos quatro atributos das 100 instâncias, e o segundo item é um vetor contendo o rótulo de cada instância.

2 Aplicação de *k-means* em um conjunto de dados

O SciKit-Learn inclui diversos algoritmos de agrupamento por meio do módulo `sklearn.cluster`. São classes com métodos de treinamento e predição. Para utilizar esses recursos, um objeto deve ser criado a partir da classe do algoritmo que se deseja implementar, configurando as propriedades do objeto para alterar os parâmetros do algoritmo (SCIKIT-LEARN, 2019a).

A biblioteca implementa o algoritmo de *k-means*. No modo padrão, é feita a escolha aleatória dos centroides (conforme quantidade especificada pelo parâmetro *k*), iniciada a classificação dos objetos mais próximos de cada centroide e feito o cálculo dos novos centroides a partir dos objetos pertencentes ao grupo. Os últimos dois passos são repetidos até que não ocorram modificações nos grupos entre as iterações. Há ainda a possibilidade de utilizar uma versão modificada do algoritmo de *k-means* conhecida como *k-means++*, que seleciona os pontos iniciais de centroide por meio de um método, parte

aleatório, parte estatístico, que evitará os piores casos de inicialização que poderiam ocorrer no algoritmo básico (ARTHUR; VASSILVITSKII, 2007).

O *k-means++* não apenas está disponível na biblioteca do SciKit-Learn como é a opção padrão do módulo *k-means*. A inicialização aleatória pode ser utilizada por meio do parâmetro `init='random'`, conforme o exemplo a seguir.



Exemplo

```
# Exemplo 4
# importação do módulo KMeans da biblioteca SciKit-Learn
from sklearn.cluster import KMeans
# alg recebe o objeto do método KMeans aleatório básico
alg = KMeans(init='random')
```

Caso seja omitido, o parâmetro `init` padrão será `init='k-means++'`. Outros parâmetros podem ser configurados, como: `n_clusters`, `n_init`, `max_iter`, `tol`, `precompute_distances`, `verbose`, `random_state`, `copy_x`, `n_jobs` e `algorithm`.

O parâmetro `n_clusters` recebe um valor inteiro e define a quantidade de grupos buscados, geralmente conhecido por parâmetro *k*. O `n_init`, também do tipo inteiro, determina a quantidade de vezes que o algoritmo será executado na íntegra com novos valores iniciais para os centroides, entretanto, apenas o melhor resultado será selecionado. O padrão é de 10 vezes (SCIKIT-LEARN, 2019b).

A quantidade de iterações máximas executadas também pode ser limitada pelo valor de `max_iter` (padrão de 300 iterações). Caso o algoritmo de *k-means* não estabilize antes desse limite, as iterações são interrompidas de qualquer forma (SCIKIT-LEARN, 2019b). Apesar de contrariar a teoria do algoritmo, essa é uma limitação prática para evitar que problemas excessivamente complexos resultem em numerosas iterações atrasando o restante da aplicação, muitas vezes, sem grande contribuição para melhorar o resultado.

A variável `tol` é um valor do tipo `float` (ponto flutuante) para determinar a tolerância da variância admitida para que seja admitida a convergência do resultado (SCIKIT-LEARN, 2019b). Idealmente, espera-se que somente uma solução seja válida, no entanto, arredondamentos e limitações de casas decimais no cálculo poderiam fazer com que pequenas variações fossem mascaradas, resultando em mudanças de agrupamento que não estabilizariam o algoritmo. Por padrão: `tol` = 0,0001.

O parâmetro `precompute_distances` pode ser definido como `auto` (padrão) ou `true/false`. Quando ativado, ele acelera o processo de cálculo das distâncias, porém pode consumir uma quantidade significativa de memória para problemas com muitos *clusters* e amostras. No modo automático (auto), a pré-computação das distâncias será feita sempre que *clusters* · amostras ≤ 12.000.000 (SCIKIT-LEARN, 2019b). Já o valor de `verbose`, de tipo inteiro, define o nível de informações retornadas pelo algoritmo durante as iterações. Se omitido, assumirá o valor zero, e nenhuma informação é retornada durante o processo. Pode ser útil para a depuração do código e para compreender com mais profundidade a forma como o algoritmo é executado (SCIKIT-LEARN, 2019b).

Para iniciar os centroides de forma aleatória, o algoritmo depende de um algoritmo gerador de valores pseudoaleatórios com base no valor inteiro de `random_state` para melhorar a condição de aleatoriedade (SCIKIT-LEARN, 2019b). Por padrão, `copy_x` recebe o valor `true`, determinando que os valores originais não sejam diretamente manipulados durante as iterações, preservando o resultado de pequenas variações durante o cálculo que poderiam atrasar o algoritmo (SCIKIT-LEARN, 2019b). Essa configuração, no entanto, afeta a quantidade de memória utilizada por demandar a cópia dos valores antes de cada iteração.

O valor de `n_jobs` permite efetuar mais de uma inicialização do *k-means* em tarefas paralelas. É um inteiro que define a quantidade de processamentos em paralelo, o padrão é apenas 1, fazendo com que a quantidade máxima permitida de processadores seja selecionada através do valor –1 (SCIKIT--LEARN, 2019b). Por fim, `algorithm` seleciona entre três formatos de cálculo das distâncias:

1. `full` — padrão dos algoritmos de *k-means* que calcula a distância de todos os pontos com relação ao centro;
2. `elkan` — converge para os mesmos resultados, mas reduz, a partir de triangulações, a quantidade de distâncias calculadas, eliminando muitas etapas desnecessárias/redundantes;

3. `auto` — recorre ao método convencional para conjuntos de dados esparsos, em que o algoritmo `elkan` implementado é incapaz de convergir, e ao método `elkan` para conjuntos de dados densos (SCIKIT-LEARN, 2019b).

Apesar da simplicidade com que é definido, o algoritmo de `k-means`, como muitos outros algoritmos, tem inúmeras limitações e parâmetros na prática, a maioria relacionada ao tempo e à taxa de processamento demandados. Na maior parte das vezes, especialmente quando estiver lidando com conjuntos de dados pequenos, as configurações se limitarão à quantidade de *clusters* (`n_clusters`) e ao valor de base para geração de pontos aleatórios (`random_state`). O código a seguir exemplifica a criação de um objeto da classe *k-means* a partir desses dois parâmetros, definindo $k = 4$ e para o `random_state`.



Exemplo

```
# Exemplo 5
# importação do módulo KMeans da biblioteca SciKit-Learn
from sklearn.cluster import KMeans
# alg recebe o objeto do método KMeans configurado para
# identificar 4 clusters. O valor 50 serve de base para o
# cálculo do ponto inicial de  cada centroide.
alg = KMeans(n_clusters=4, random_state=50)
```

Uma vez criado o objeto com as devidas configurações, o conjunto de dados deve ser fornecido através da função `fit` que fará o treinamento do modelo. Nela, os parâmetros `X` e `sample_weight` (opcional) correspondem, respectivamente, aos dados de treinamento, em forma de vetor ou matriz, e aos pesos que devem ser dados para cada observação. O código a seguir faz o treinamento de dados com 100 amostras, dois atributos separados em três grupos de centro diferentes produzidos a partir do método `make_blobs e`.



Exemplo

```
# Exemplo 6
# carrega bibliotecas e módulos necessários
from sklearn import datasets as ds
from sklearn.cluster import KMeans
import pandas as pa
import seaborn as sb
import matplotlib.pyplot as pp
# distribuição aleatória: 100 objetos, 4 grupos, 2 carac-
terísticas desvio padrão de 1,3
X, y = ds.make_blobs(n_samples=100, n_features=2, cen-
ters=4, cluster_std=1.3)
# inicializa e treina um algoritmo de K-Means
km = KMeans(n_clusters=4, random_state=50)
km.fit(X)
prevs = km.predict(X)
# DataFrames para o conjunto de dados original com seu
# atributo alvo e outro quadro com o atributo alvo previsto
# a partir do K-Means
xDF = pa.DataFrame(data=X, columns=['Attr1', 'Attr2'])
pDF = xDF.copy()
xDF['Target'] = y
pDF['Target'] = prevs
# divisão da figura em duas colunas para plotar os dois
# conjuntos
fig, axes = pp.subplots(1, 2, figsize=(12,4))
sb.scatterplot(data=xDF, x='Attr1', y='Attr2', hue='Target',
palette='rainbow', ax=axes[0])
sb.scatterplot(data=pDF, x='Attr1', y='Attr2', hue='Target',
palette='rainbow', ax=axes[1])
```

Com o auxílio das bibliotecas Pandas e Seaborn, são criados os `DataFrame` e os gráficos de dispersão (`scatterplot`), respectivamente. Uma vez que o método `make_blobs` retorna uma tupla, o resultado é desmembrado em duas variáveis: *x* para os atributos previsores e *y* para o atributo alvo. O método `predict` cria um vetor para o atributo previsor a partir dos *clusters* descobertos pelo método `fit` anteriormente. Os quadros (`DataFrame`) facilitam tanto a apresentação dos dados quanto sua utilização em outras funções do `Seaborn`, quando é possível fornecer os índices de característica (*x* e *y*) e o índice que representa as classes pelo atributo `hue`. A Figura 1 apresenta um comparativo entre os dados originais e suas classificações e os dados previstos pelo *k-means*.

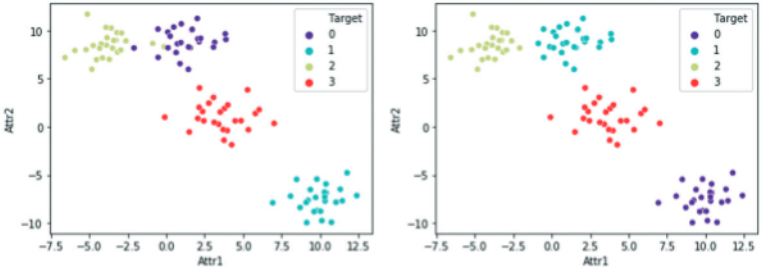


Figura 1. Comparativo entre os dados originais e suas classificações (à esquerda) e os dados previstos pelo *k-means* (à direita).

Como é possível perceber na Figura 1, *clusters* parecidos foram detectados, contudo, devido a pontos de sobreposição (imagem da esquerda), alguns objetos dos grupos 2 e 3 foram invertidos na imagem da direita. Também devido a isso, não foram atribuídos exatamente os mesmos rótulos para cada grupo, porque cada função utilizou os objetos em ordens diferentes. Os nomes dados, no entanto, não possuem qualquer relevância, não é atribuição dos algoritmos de agrupamento atribuir um rótulo, mas encontrar grupos que possam ser rotulados. Nesse caso, pode-se perceber que o treinamento, apesar de possuir imprecisões, foi muito eficiente, apresentando poucas divergências de *clusters*.

3 Agrupamento e avaliação

A fim de compreender o algoritmo de agrupamento, um novo exemplo utilizando o conjunto de dados `íris` é criado no código a seguir.



Exemplo

```
# Exemplo 7
# importação das bibliotecas e seleção dos módulos a serem
# utilizados
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
import pandas as pa
import seaborn as sb
import matplotlib.pyplot as pp
import numpy as np
# importação e separação dos dados
iDS = load_iris()
iData, iTarget, iTarget_Names, _, iFeature_Names, _ =
iDS.values()
# formatação dos dados em quadro da biblioteca pandas para
# facilitar a visualização
iDF = pa.DataFrame(data=iData, columns=iFeature_Names)
iDF_Target = iDF.copy()
iDF_Target['specie'] = iTarget_Names[iTarget]
# visualizar a sobreposição das espécies entre todas as
# combinações de pares de características
sb.pairplot(data=iDF_Target, hue='specie')
```

A tupla retornada por `load_iris()` é desmembrada em seis variáveis:

- `iData` — matriz contendo os valores de cada instância de cada atributo previsor;
- `iTarget` — vetor contendo os valores de atributo alvo para cada instância;

- `iTarget_Names` — vetor com os nomes dados a cada uma das três espécies rotuladas;
- `iFeature_Names` — vetor com o nome dado aos atributos previsores;
- outros dois itens da tupla recebem um nome qualquer, pois não há interesse em seus usos, referem-se a uma *string* contendo a descrição e a outra contendo o caminho para o arquivo.

Os dados de entrada e saída (atributos previsores e atributo alvo) são utilizados para construir um `DataFrame`. A coluna alvo recebe o nome `specie` (espécie). O `DataFrame` é utilizado na plotagem da distribuição dos dados aos pares de atributos (função `pairplot`), sempre classificados segundo a coluna `specie`, conforme especificado pelo parâmetro `hue`. O resultado pode ser visto na Figura 2.

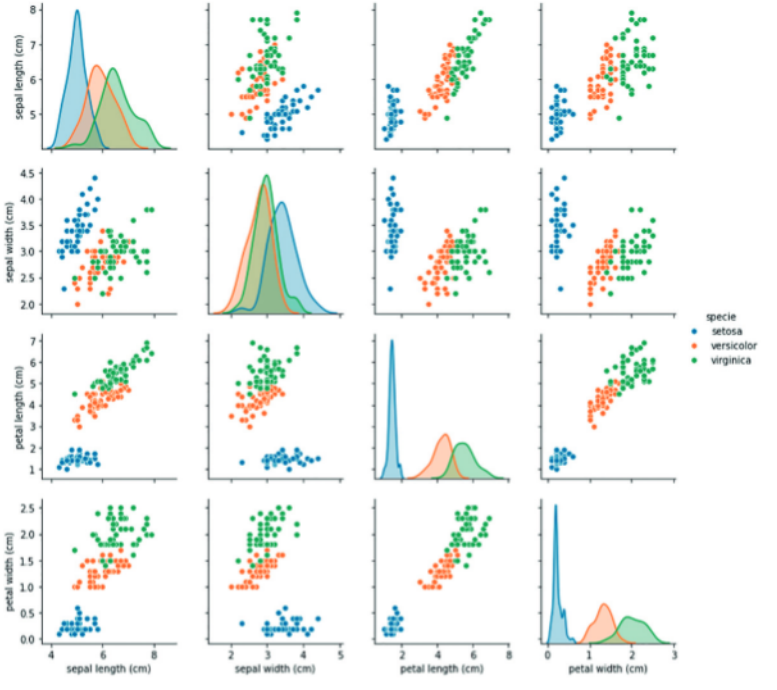


Figura 2. Gráficos de distribuição das instâncias selecionando pares de cada um dos atributos.

Qualquer combinação de características pode ser utilizada na tarefa de agrupamento. O resultado, no entanto, será muito diferente de uma para outra. Todos os gráficos da Figura 2 possuem algum nível de sobreposição, algumas combinações parecem ter maiores distinções entre os agrupamentos do que outras. Poderíamos descartar o cruzamento entre as duas características das sépalas (*sepal*) apenas visualizando os gráficos. Para esse exemplo, optou-se pela combinação *sepal length* e *petal width* por relacionar características bem diferentes e aparentar uma distribuição com boa separação de espécies. No entanto, outras combinações poderiam resultar em agrupamentos ainda melhores.

O exemplo a seguir apresenta a seleção dos atributos que serão utilizados, respectivamente, a primeira (índice 0) e a quarta (índice 3) coluna da matriz, como foi possível ver nos gráficos da Figura 2. Mais uma vez é criado um `DataFrame` e adicionada a coluna dos rótulos originais ao novo `DataFrame`.



Exemplo

```
# Exemplo 8
# seleção das duas características
iData_SL_PW = iData[:, [0,3]]
iDF_SL_PW   =   pa.DataFrame(data=iData_SL_PW,
columns=[iFeature_Names[0], iFeature_Names[3]])
iDF_SL_PW_T = iDF_SL_PW.copy()
iDF_SL_PW_T['specie'] = iDF_Target['specie']
# distribuição após seleção dos atributos sem rótulos
# definidos
prevFeatureNames = iDF_SL_PW_T.keys()
sb.scatterplot(data=iDF_SL_PW_T, x=prevFeatureNames[0],
y=prevFeatureNames[1])
```

O gráfico da Figura 3 mostra novamente a distribuição dos dados segundo o par de características selecionadas, porém, sem qualquer rótulo ou agrupamento que as separe.

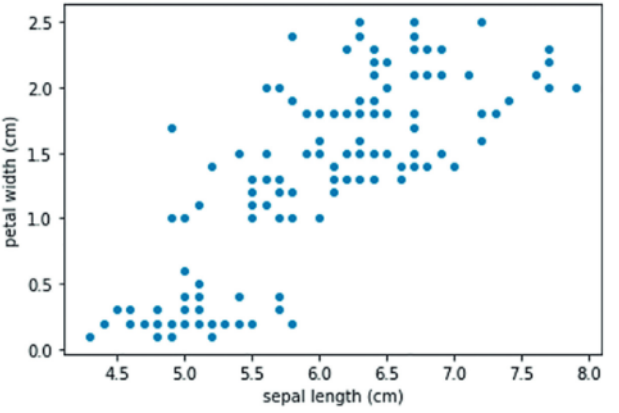


Figura 3. Distribuição dos dados sem *clusters* formados.

O algoritmo de *k-means* é configurado para buscar a mesma quantidade de *clusters* que a quantidade de espécies esperada, a previsão de agrupamento é treinada e feita segundo os *clusters* encontrados para o mesmo conjunto de dados utilizados no treinamento. O vetor resultante é atribuído ao novo `DataFrame`, agora com os dados das duas características de entrada, mas como atributo alvo os valores previstos pelo algoritmo. A seguir, é apresentado o código desse procedimento.

Exemplo

```
# Exemplo 9
# quantidade de clusters será a mesma da quantidade de
# espécies do conjunto de dados
k = len(iTarget_Names)
# configuração do algoritmo e treinamento
alg = KMeans(n_clusters=k, random_state=50)
alg.fit(X=iData_SL_PW)
# construção de novo quadro considerando os agrupamentos
# previstos pelo K-Means
prevDataFrame = iDF_SL_PW.copy()
prevDataFrame['specie'] = alg.predict(iData_SL_PW)
prevDataFrame
```

O gráfico da Figura 4 serve de base para que os grupos encontrados sejam então rotulados pelo usuário. É possível observar que a ordem detectada não é a mesma, mas é visível qual *cluster* se refere a qual espécie.

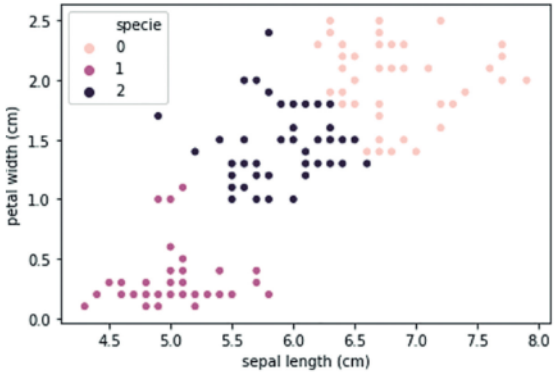


Figura 4. *Clusters* encontrados.

Na Figura 4, é possível notar três *clusters* em posições equivalentes aos formados pelas espécies *setosa*, *virginca* e *versicolor*, como já observado no `pairplot` entre as características selecionadas da Figura 4.



Exemplo

```
# Exemplo 10
translateDic = { 0: 'virginica', 1: 'setosa', 2: 'versicolor' }
prevDataFrame['specie'] = [translateDic[s] for s in
prevDataFrame['specie']]
fig, axes = pp.subplots(1, 2, figsize=(12,4))
fig1 = sb.scatterplot(data=iDF_SL_PW_T,
x=prevFeatureNames[0], y=prevFeatureNames[1], hue='specie',
ax=axes[0])
fig2 = sb.scatterplot(data=prevDataFrame,
x=prevFeatureNames[0], y=prevFeatureNames[1], hue='specie',
ax=axes[1])
```

O exemplo anterior faz a tradução e termina por exibir um comparativo entre os *clusters* originais e os previstos pelo modelo (Figura 5).

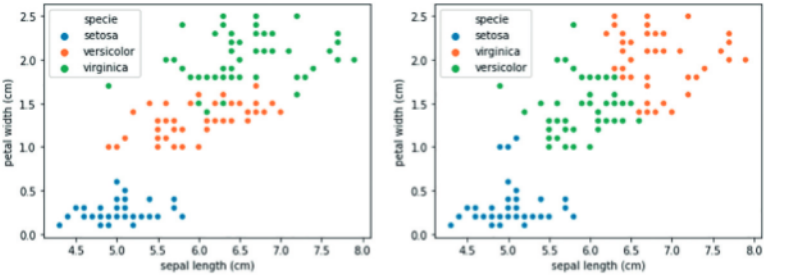


Figura 5. Comparativo entre os *clusters* originais (à esquerda) e os previstos pelo *k-means* (à direita).

Na Figura 5, nota-se que as cores são diferentes para cada um dos *clusters*, motivo que levou à criação do dicionário de tradução, já que a ordem dos *clusters* está invertida. Por essa razão, é possível imaginar que existem instâncias agrupadas em *clusters* diferentes em cada exemplo, e uma rápida observação denunciaria isso entre as espécies *virginica* e *versicolor*. As flores que, originalmente, eram classificadas como *virginica*, mas apresentavam características de comprimento de sépala e largura de pétala que as inseriam dentro do *cluster* da espécie *versicolor*, não puderam ser agrupadas de modo correto pelo algoritmo de *k-means* devido a essa sobreposição na distribuição. Também é possível constatar a quantidade de erros ao comparar quais instâncias estão agrupadas em *clusters* diferentes entre cada `DataFrame` (original e previsto), contabilizando o tamanho do vetor encontrado através da função `len` que retornará um total de 26 instâncias com *clusters* divergentes.



Exemplo

```
# Exemplo 11
len(prevDataFrame[iDF_SL_PW_T['specie']    !=
prevDataFrame['specie']])
cm    =    confusion_matrix(iDF_SL_PW_T['specie'],
prevDataFrame['specie'])
cmtx = pa.DataFrame(
           cm,
          index=('Real: ' + pa.DataFrame(iTarget_Names))
[0],
           columns=('Prev: ' + pa.DataFrame(iTarget_Na-
mes))[0]
        )
cmtx,    accuracy_score(iDF_SL_PW_T['specie'],
prevDataFrame['specie'])
```

O exemplo anterior, além de apresentar o algoritmo para calcular os erros, também utiliza a função `confusion_matrix` para apresentar uma matriz de confusão relacionando os valores esperados e os obtidos para cada grupo, como apresentado no Quadro 3, além do percentual de acerto segundo o método `accuracy_score`.

Quadro 3. Matriz de confusão

	Prev: *setosa*	**Prev: *versicolor***	**Prev: *virginica***
Real: *setosa*	50	0	0
Real: *versicolor*	4	39	7
Real: *virginica*	0	15	35

Entretanto, a matriz de confusão e a quantidade de erros só podem ser utilizadas porque os rótulos corretos de cada espécie são conhecidos na biblioteca SciKit-Learn. Na maioria das situações, o agrupamento é utilizado em conjuntos de dados em que se pretende aprender estruturas ou características relacionadas para, só então, determinar a possibilidade de se rotular algum grupo.

Outras métricas devem ser utilizadas em situações em que os grupos originais não são conhecidos. O exemplo a seguir calcula a silhueta média dos *clusters*, uma das métricas mais utilizadas na avaliação de agrupamentos por *k-means*. Ambos buscam por grupos que possuem pouca ou nenhuma sobreposição.



Exemplo

```
# Exemplo 12
from sklearn.metrics import silhouette_score
prevs = alg.predict(iData_SL_PW)
silhouette_score(iData_SL_PW, prevs)
```

Para a silhueta, é encontrado um valor de 0,5, que é um bom valor para um índice que varia de –1 a 1. Quanto mais baixo esse número, maior a quantidade de instâncias que poderiam ser agrupadas em outro *cluster*. Outro indicador disponibilizado pela biblioteca do SciKit-Learn é o índice de Calinski-Harabasz, que calcula e avalia a taxa de dispersão entre os *clusters* pela dispersão entre os membros de cada um deles. Veja o exemplo a seguir.



Exemplo

```
# Exemplo 13
from sklearn import metrics
from sklearn.metrics import pairwise_distances
metrics.calinski_harabasz_score(iData_SL_PW, prevs)
```

Quanto maior for o índice de Calinski-Harabasz, melhor será o agrupamento. No entanto, esse valor fará mais sentido quando comparado com diferentes agrupamentos, podendo determinar qual é o maior deles e, portanto, o mais eficiente. A critério de comparação, quando testado o treinamento do *k-means* para valores de $k = 2$ e $k = 4$, teríamos, respectivamente, índices iguais a 318,25 e 323,32, enquanto que, para $k = 3$, conforme os exemplos executados, o índice seria de 350,24, ou seja, o maior entre eles conforme o esperado, já que sabemos que três *clusters* existem no conjunto de dados original.



Referências

ARTHUR, D.; VASSILVITSKII, S. K-means++: the advantages of careful seeding. *In*: ACM-SIAM SYMPOSIUM ON DISCRETE ALGORITHMS, 18., 2007, New Orleans. *Proceedings* [...]. Philadelphia: SIAM, 2007, p. 1027–1035. Disponível em: http://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf. Acesso em: 28 fev. 2020.

SCIKIT-LEARN. *Clustering*. c2019a. Disponível em: https://scikit-learn.org/stable/modules/clustering.html#k-means. Acesso em: 28 fev. 2020.

SCIKIT-LEARN. *sklearn.cluster.KMeans*. c2019b. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html#sklearn.cluster.KMeans. Acesso em: 28 fev 2020.

Leituras recomendadas

ANACONDA INC. *Anaconda distribution*. c2020. Disponível em: https://www.anaconda.com/distribution/#download-section. Acesso em: 28 fev. 2020.

FACELI, K. *et al. Inteligência artificial*: uma abordagem de aprendizado de máquina. Rio de Janeiro: LTC, 2011.

JAMES, G. *et al. An introduction to statistical learning*: with application in R. New York: Springer, 2013.

WITTEN, I. H.; FRANK, E. *Data mining*: practical machine learning tools and techniques. 2nd ed. Boston: Morgan Kaufmann, 2005.



Fique atento

Os *links* para *sites* da *web* fornecidos neste livro foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS